

# Explainability Analysis of a Chess Position Classification Neural Network

Artem Okhten  
MTH 4326 – Explainable AI

Spring 2026

## Abstract

In this project I study counterfactual explanations for convolutional neural network that classifies chess positions. The model predicts one of three classes: Black-better, Draw/Equal, or White-better. Dataset comes from Lichess PGN games. For each game, I extract one position after 20 full moves and convert board into  $13 \times 8 \times 8$  tensor. The tensor has 12 channels for pieces and one channel for whose turn it is. Labels are made using Stockfish evaluations, not final game results, because I want model to learn quality of current position.

The goal is not to build strongest chess engine. The CNN is used as model that can be explained. First, I use gradient saliency map as baseline explanation method. This shows which squares model is locally sensitive to. Then I use counterfactual search to test what small board changes are enough to change model prediction. The main counterfactual method removes one non-king piece at a time. I measure size of change using material distance and number of changed squares.

The CNN reached 63.73% test accuracy, compared with 44.60% majority-class baseline on test set. This means model learned some chess evaluation structure, even though it is not perfect evaluator. In counterfactual experiment, one-piece removal changed model prediction in 76.20% of sampled test positions. Many successful counterfactuals came from low-material pieces, especially pawns. This suggests that model decision boundary is often close to original position. Overall, counterfactual explanations give another way to inspect model beyond accuracy and saliency maps.

## 1 Introduction and Motivation

In this project I analyze counterfactual explanations for neural network that classifies chess positions. Instead of only asking which squares or pieces are important, I ask what small change to position is enough to change model prediction.

The model is not intended to be strongest chess engine. It is CNN trained on board tensors so that its predictions can be inspected. I want to understand where its decision boundaries are and whether counterfactual changes make chess sense, for example material balance, dependence on individual pieces, or positional structure.

This project is mainly empirical, so report is connected to outputs from Jupyter notebook. The code itself stays in notebook, while this report explains method, results, and interpretation. The

focus is not only on model accuracy. Accuracy is first check that model learned something. Main part is explanation validity, counterfactual distance, and stability of the model decisions.

## 1.1 Research Questions

This project is based on three questions:

1. Can small CNN trained on chess position tensors learn enough structure to be useful for explanation?
2. Do gradient saliency maps and counterfactual explanations show different parts of model behavior?
3. How stable are model predictions when small piece-removal changes are made?

These questions matter because explainability is not only about making heatmap. Useful explanation should also test whether changing input actually changes model decision.

## 2 Dataset and Model

The dataset consists of chess positions extracted after fixed number of moves. Each position is represented as a  $13 \times 8 \times 8$  tensor:

- **12 piece channels:** White and Black pawns, knights, bishops, rooks, queens, and kings.
- **1 turn channel:** shows whose turn it is to move.

The labels are based on Stockfish evaluation of the position:

- **Class 0 (Black better):** Evaluation  $< -1.0$
- **Class 1 (Equal / balanced):**  $-1.0 \leq \text{Evaluation} \leq +1.0$
- **Class 2 (White better):** Evaluation  $> +1.0$

I used Stockfish with short time limit, 0.1 seconds per position, to keep preprocessing possible on my computer. Because of this, labels should be understood as approximate engine labels, not full deep Stockfish analysis.

For final experiment, I used 10,000 positions extracted from Lichess PGN games. The positions are extracted after 20 full moves, so model looks at early middlegame structure instead of final game result. In the code, `max_plies=40` means that position is extracted after 20 full moves. One ply is a half-move, so 40 plies means White and Black both made 20 moves.

After dataset is prepared, I train simple Convolutional Neural Network (CNN) to classify these positions. The CNN is not meant to be strongest evaluator. It is used as model that will later be explained with saliency and counterfactual methods.

## 2.1 Dataset Distribution

The final dataset contains 10,000 positions. Class distribution is not perfectly balanced: White-better positions appear most often, Black-better positions are second, and Draw/Equal positions appear less often. However, all three classes are still represented well in training, validation, and test sets.

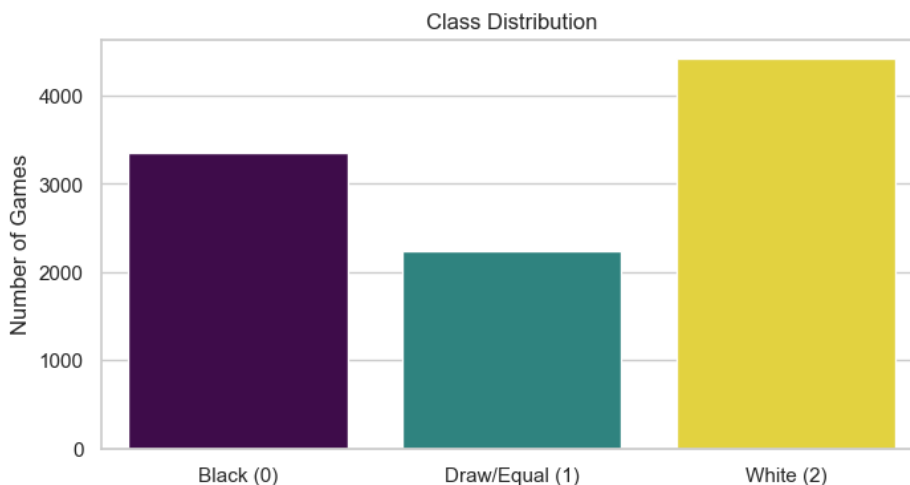


Figure 1: Class distribution for the 10,000 extracted chess positions.

The majority-class baseline for entire dataset is 44.15%. This is important because model should not be compared only to random guessing. It also should be compared to simple baseline that always predicts most common class. The majority-class baseline gives more realistic lower bound for model performance.

## 3 Methodology Overview

The experiment follows five main steps:

1. **Position extraction:** Parse Lichess PGN games and extract one board position after 20 full moves.
2. **Label generation:** Evaluate each extracted position with Stockfish and assign it to Black-better, Draw/Equal, or White-better class.
3. **Model training:** Train small CNN on the  $13 \times 8 \times 8$  board tensors.
4. **Baseline explanation:** Use gradient saliency to see which board squares affect predicted class score.
5. **Counterfactual analysis:** Remove one non-king piece at a time and check if prediction changes.

The main evaluation is not only classification accuracy. Accuracy is used just to check that model learned some signal from data. After that, I also measure explanation validity, counterfactual distance, and stability across predicted classes.

### 3.1 Experimental Design

The CNN is trained mainly to create a model that can be explained. Because of this, I do not tune model very aggressively for maximum accuracy. Instead, I use small architecture that is easy to inspect and fast enough to train locally. This makes explanation analysis more important than predictive performance alone.

The counterfactual experiment is run on held-out test positions, not on training set. This is important because explanations should be tested on positions that model did not see directly during training. If counterfactuals are only tested on training data, then results can look better than they actually are.

## 4 Model Training and Performance

The training loss decreases steadily over epochs, which shows that CNN learned patterns from board tensor representation. Validation accuracy also improves over training and finishes around 61.60%, so model does not look like it is only memorizing training data.

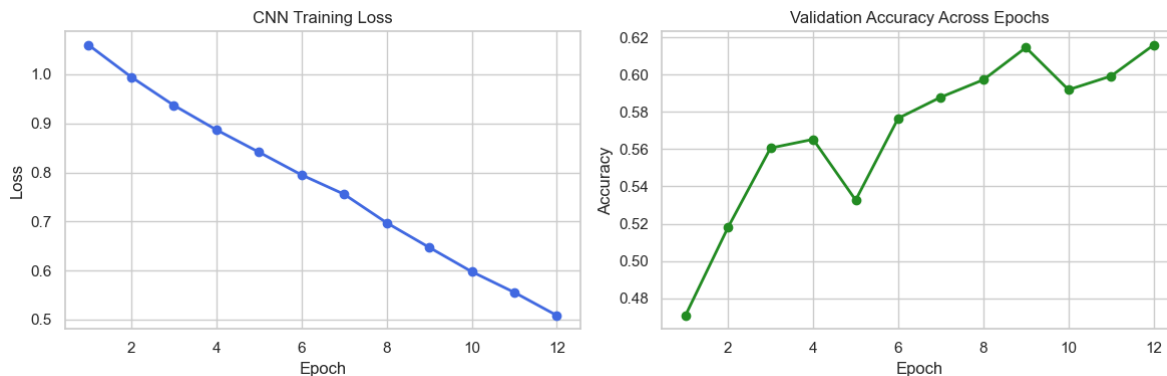


Figure 2: CNN training loss and validation accuracy across epochs.

The test accuracy is 63.73%, while test majority-class baseline is 44.60%. This gives improvement of 19.13 percentage points over simple baseline. The model is not perfect chess evaluator, but it performs clearly better than random guessing and better than baseline that always predicts most common class.

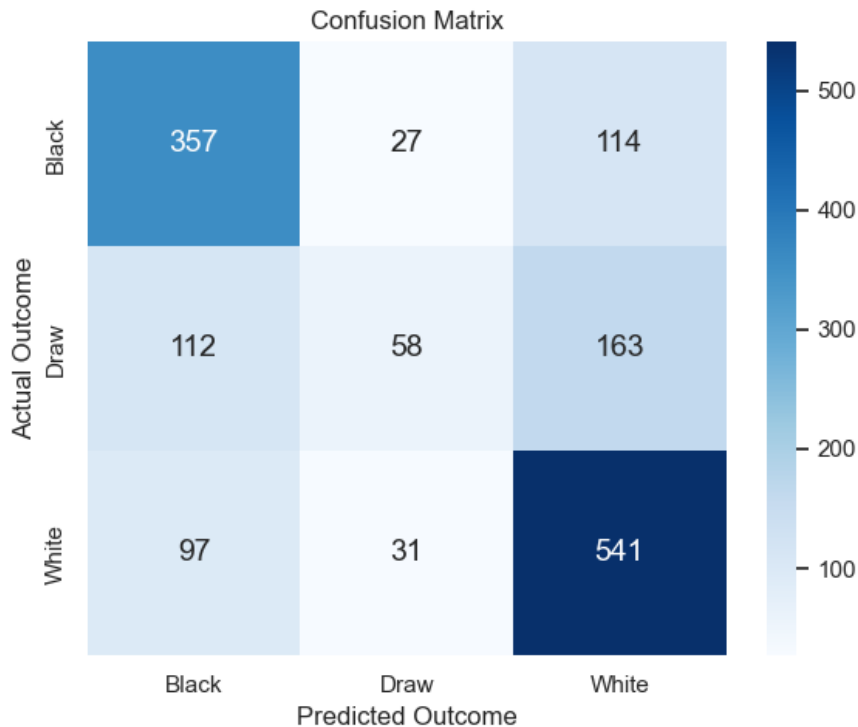


Figure 3: Confusion matrix for the CNN on test set.

The classification report and confusion matrix show that model performs strongest on White-better and Black-better positions. White class has high recall, and Black class is also identified reasonably well. Draw/Equal is harder for the model, with lower recall. This makes sense because Draw/Equal positions are in the middle between the two advantage classes, and positions close to Stockfish threshold can be strategically similar.

For this project, this performance is acceptable. The CNN learned enough evaluation structure to be useful as model for explanation. Because of that, counterfactual analysis is not just testing random noise, but model decisions that came from learned chess patterns.

## 5 Baseline Explanation: Gradient Saliency

Before applying counterfactual explanations, I first compute simple gradient-based saliency map as baseline explanation method. This is included because project is about Explainable AI, and baseline method helps compare counterfactual explanations with more standard importance-based explanations.

The saliency map measures how sensitive predicted class score is to each input square. In this project, saliency values are summed across 13 tensor channels to create one  $8 \times 8$  board-level importance map. This does not prove causality, but it gives first view of which board squares CNN is reacting to.

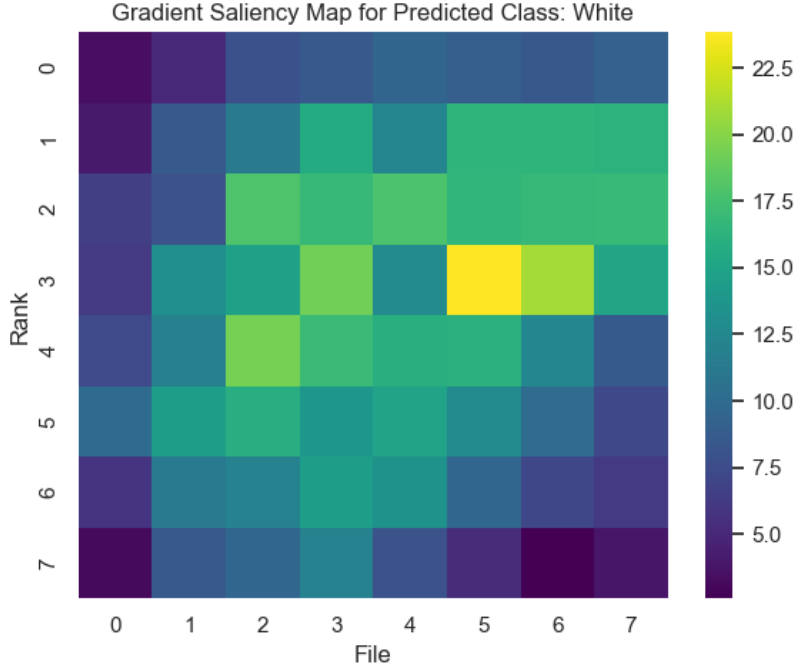


Figure 4: Gradient saliency map for one selected test position.

The saliency map gives board-level view of where CNN is most sensitive for one selected position. Brighter squares mean that changes to those input locations would have larger local effect on predicted score.

However, this explanation is only local and gradient-based. A highlighted square does not always mean that changing this square will actually flip model prediction. This is why counterfactual explanations are useful in next section. They test whether a concrete intervention, such as removing one non-king piece, actually changes predicted class.

## 6 Counterfactual Explanation Method

The main explainability method in this project is counterfactual search. A counterfactual explanation tries to answer simple question: what minimal change to input would make model predict different class?

In proposal, I described three possible levels of counterfactual generation:

- **Level 1:** remove one non-king piece.
- **Level 2:** relocate one piece to nearby empty square.
- **Level 3:** apply true legal-move counterfactuals.

For implemented experiment, I focus on Level 1 counterfactuals. This is simplified version of original plan. I chose it because it is simple, interpretable, and directly connected to chess material

value. Removing piece is not always legal chess move, so I treat it as intervention-based sensitivity test rather than fully realistic chess variation.

The method still follows main goal of proposal: it finds small perturbations that change predicted class, measures distance of these perturbations, and checks whether resulting explanations are stable across different model predictions.

## 7 Counterfactual Experiments and Results

For counterfactual analysis, I sampled up to 500 test positions. This gives more stable estimate of flip rate and counterfactual distance than using only few examples. The counterfactual search is applied only to test data, so explanations are checked on positions that model did not directly train on.

### 7.1 Counterfactual Validity and Piece-Level Patterns

The flip rate is used as simple validity metric. Counterfactual is valid if intervention changes model prediction. In this experiment, method was able to find successful one-piece intervention for 76.20% of sampled test positions.

At first this looks like strong result because counterfactual search often finds valid examples. However, this result should be interpreted carefully. A very high flip rate also means that model predictions are fragile. In many cases, removing only one piece, often a pawn, was enough to move prediction to another class. A strong chess evaluator should not change its evaluation too easily after removal of low-value piece unless that piece is tactically or positionally critical.

Because of this, high flip rate is not only evidence that counterfactual method works. It is also evidence that this CNN has unstable decision boundaries. This is consistent with model performance: the CNN performs above baseline, but it is still far from strong chess engine. Therefore, counterfactuals are useful here because they reveal weakness of trained model, not only its reasoning pattern.

In my run, most common counterfactual pieces were White pawns, Black pawns, bishops, queens, and knights. The top changes were White Pawn, Black Pawn, White Bishop, White Queen, and White Knight. This matters because pawn removals have smallest material distance, but they still changed many predictions. At same time, some queen removals also appeared, so not every counterfactual is only small material change.

This does not mean that every pawn is decisive in real chess. Instead, it means that for this trained CNN, small pawn-level perturbations were often enough to cross learned boundary between classes. Counterfactual explanations therefore show model sensitivity that may not be obvious from accuracy alone.

## 7.2 Counterfactual Distance

The distance distribution shows how large smallest successful intervention usually was. Smaller distances mean that prediction can be changed by removing low-value piece, while larger distances mean that model prediction is more resistant and needs larger material change.

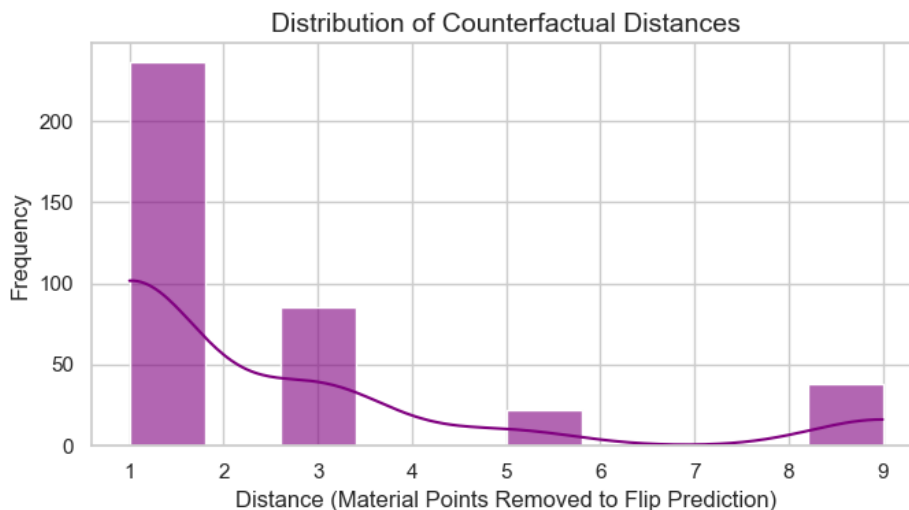


Figure 5: Distribution of material distance for successful counterfactuals.

In my run, distribution is mostly concentrated at 1 and 3 material points. There are also some larger-distance counterfactuals around 5 and 9 points, but they are less common. This supports idea that many model predictions are fragile under small local interventions. If model prediction changes after one-point material change, then learned decision boundary is close to original position.

## 8 Stability Analysis

One important part of reliable XAI method is explanation stability. Here, I check whether positions with same original predicted class need similar counterfactual changes to alter model prediction. This is not perfect measure of similar positions, but it gives simple way to see whether same type of model decision has similar counterfactual distance.



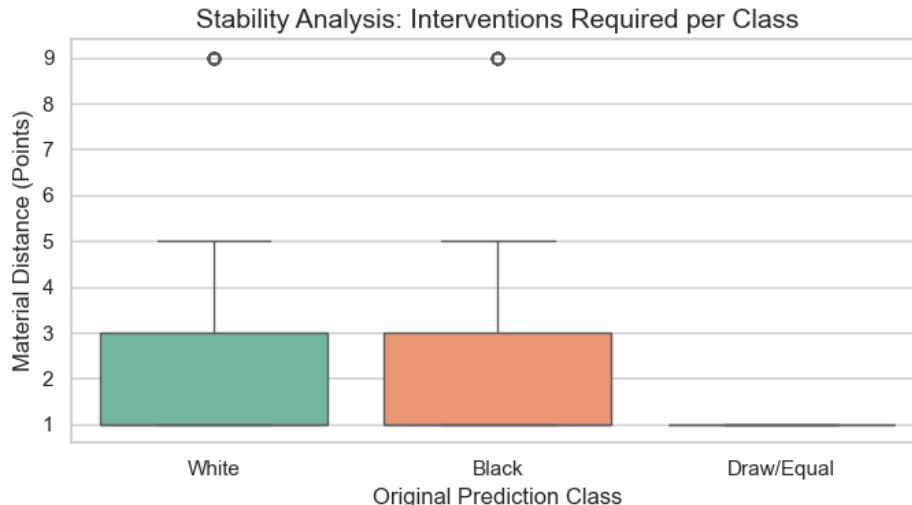


Figure 6: Counterfactual material distance grouped by original predicted class.

The stability analysis groups counterfactuals by original predicted class. All three classes show low average counterfactual distances. Draw/Equal has lowest average distance, about 1.00, which means these predictions are very easy to flip when one valid counterfactual is found. Black has average distance around 2.46, and White has average distance around 2.81. These values are still low because they are below major-piece material values.

The number of valid counterfactuals is also different by class. In my run, White positions produced largest number of valid counterfactuals, then Black positions, while Draw/Equal produced fewer. This suggests that model boundary around White and Black predictions is often close enough that many one-piece removals can change prediction.

Overall, stability results show that explanations are valid, but not highly stable. Many predictions can be changed by small interventions, especially pawn removals. This is important finding because it suggests that CNN decision boundaries are relatively close to many test positions.

## 8.1 Confidence-Based Stability

I also compare counterfactual distance with model original prediction confidence using scatterplot. If higher-confidence predictions need larger interventions, then confidence is related to stability. However, if low-distance counterfactuals also happen at high confidence, then model can be confident but still fragile under small input changes.

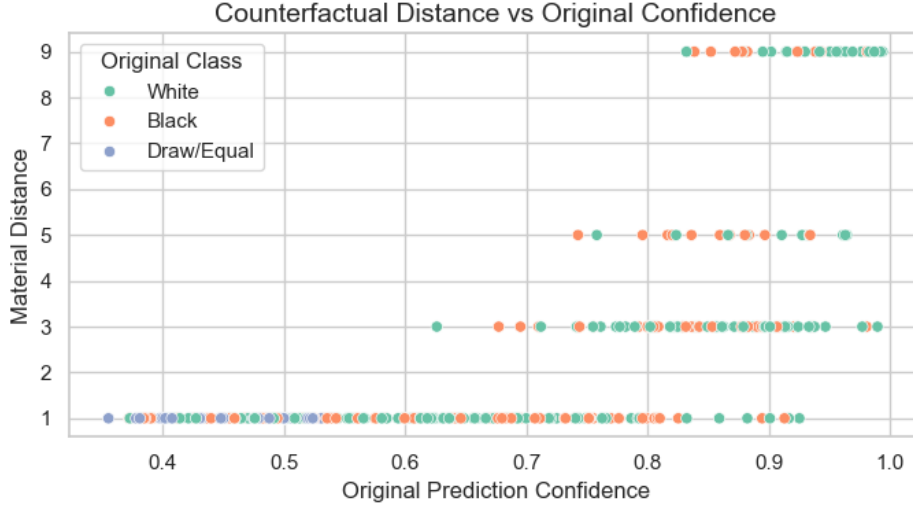


Figure 7: Counterfactual material distance compared with original prediction confidence.

The confidence plot shows that low-distance counterfactuals can happen across different confidence levels. This means model can sometimes be confident but still fragile under small input changes. Larger material-distance counterfactuals appear more often at higher confidence, but low-distance changes are still common.

## 9 Summary of Empirical Findings

| Component               | Main Result                                  | Interpretation                                                            |
|-------------------------|----------------------------------------------|---------------------------------------------------------------------------|
| Dataset                 | 10,000 Lichess positions after 20 full moves | Realistic dataset for local XAI experiment                                |
| Model                   | Test accuracy = 63.73%, baseline = 44.60%    | CNN learned meaningful but not perfect evaluation structure               |
| Baseline explanation    | Gradient saliency map                        | Shows local sensitivity over board squares                                |
| Counterfactual validity | Flip rate = 76.20% on 500 test positions     | Many valid counterfactuals found, but also shows fragile model boundaries |
| Distance metric         | Most counterfactuals have distance 1 or 3    | Model is sensitive to low-material changes                                |
| Stability               | Average distances are low across classes     | Predictions are not highly stable under small interventions               |

Table 1: Summary of the main empirical findings.

Overall, table shows that model learned enough structure to be explained, but its decisions are still not fully stable. Accuracy is only first check. More important result for this project is that small counterfactual changes can often move prediction to another class.

## 10 Limitations and Conclusion

One limitation of this project is that Level 1 counterfactuals do not follow full chess rules. Removing a piece is not legal chess move in real game, but it is still useful as sensitivity test for model.

Another limitation is that distance metric is based mostly on material value. This is easy to understand in chess terms, but it does not fully measure positional importance. For example, pawn near promotion or key defender can be more important than its basic material value.

In conclusion, this project analyzed chess position CNN using counterfactual explanations. The model was trained on Lichess positions represented as  $13 \times 8 \times 8$  tensors, with labels generated from Stockfish evaluations after 20 full moves. The CNN reached 63.73% test accuracy, which is clearly above 44.60% test majority-class baseline. This was enough to use it as model for explanation.

The baseline saliency map showed which board squares had high local gradient sensitivity, but it did not directly show whether changing those squares would alter prediction. Counterfactual explanations addressed this problem by testing concrete interventions. The main implemented method removed one non-king piece at a time and checked whether predicted class changed.

The main empirical finding is that many predictions were changed by low-distance interventions, especially pawn removals. The counterfactual flip rate was 76.20%, which means that one-piece removal often changed model output. This is useful for XAI because it gives many valid counterfactual examples, but it also shows important weakness of the CNN. The model is much more fragile than a real chess evaluator should be. In many positions, its prediction boundary is close enough that removing a low-value piece can move the model to another class.

In other words, CNN learned meaningful chess evaluation patterns, but its decisions are still not very stable. Counterfactual explanations are valuable here exactly because they expose this fragility. They show not only what the model may depend on, but also where model is unreliable.

Future work could extend this project by using legal-move counterfactuals, relocation-based counterfactuals, larger training data, and more advanced CNN architecture.